

STAT 4710/5710: Homework 3

David Dai

Contents

Instructions	1
1 Case study: logistic regression for home loan (45 points)	3
1.1 Data import (4 points)	3
1.2 Data exploration and visualization (4 points)	4
1.3 Univariate logistic regression (23 points)	5
1.3.1 Understanding logistic regression (13 points)	5
1.3.2 Univariate logistic regression on the full data (10 points)	7
1.4 Multivariate logistic regression (14 points)	8
2 Case study: linear regression for housing prices (55 points)	11
2.1 Exploratory data analysis (10 points)	11
2.2 Predictive modeling preparation (1 point)	14
2.3 Ordinary least squares (6 points)	14
2.4 Ridge regression (13 points)	18
2.5 Lasso regression (10 points)	20
2.6 Evaluation and comparison (15 points)	22

Instructions

Materials and collaboration

The policy on allowed materials and collaboration is as stated on the Syllabus:

“Students are permitted to work together on homework assignments, but must write up and submit solutions individually. In particular, students may not copy each others’ solutions. Students may consult all course materials, textbooks, the internet, or AI tools (e.g. ChatGPT) to complete their homework. Students may not use solutions to problems that may be available online and/or from past iterations of the course. For each homework, students must disclose all classmates with whom they collaborated, which AI tools they used, and how they used them. Failure to do so will result in a 10-point penalty.”

In accordance with this policy,

Please disclose all classmates with whom you collaborated: None

Please disclose which AI tools you used, and how you used them: ChatGPT in debugging r code and creating visualizations.

Failure to answer the above questions will result in a 10-point penalty.

Writeup

Use `homework-3-template.Rmd` as a starting point for your writeup, adding your solutions after `***Solution***`. Add your R code using code chunks and add your text answers using **bold text**. Consult the [preparing reports guide](#) for guidance on compilation, creation of figures and tables, and presentation quality. If the instructions ask you to “print a tibble”, you should print the tibble directly.

Programming

We encourage you to follow the **tidyverse** paradigm for data visualization, manipulation, and wrangling, as in the first homework, to practice tidyverse workflows (e.g., `filter`, `mutate`, `group_by`, `summarize`, `join`, `ggplot2`).

Students who strongly prefer to use Python for homework may do so. If you choose this option, you are **required** to present both code chunks and the corresponding outputs/interpretations in your PDF. In addition, it’s recommended that you:

- Use **pandas** for data manipulation (preferred, since it parallels `dplyr`).
- Use **plotnine** for visualization (preferred, since it parallels `ggplot2`). If using `seaborn/matplotlib`, you must still include the required plot elements (labels, 45° line, smoothing curve, facets, etc.).

```
library(tidyverse)
library(splines)
library(glmnetUtils)
```

Code completion. In some problems, the task is to complete an incomplete code chunk provided. In those problems, in the template, there is `eval = FALSE` in the chunk header to prevent evaluation errors. You should remove `eval = FALSE` after you have completed the code to enable evaluation and printing of results.

Grading

The point value for each problem sub-part is indicated. There are 100 points possible on this homework, evaluated based on the correctness of the numerical results and plots. When a problem asks for a plot:

- **Axes and labels:** Always include informative axis labels and, if appropriate, titles.
- **Required elements:** If the instructions ask for a 45° line, smoothing curve, or facets, these must be included.
- **Clarity:** Points may be deducted if plots are cluttered, unreadable, or missing labels.

Submission

Compile your writeup to PDF and submit to Gradescope.

1 Case study: logistic regression for home loan (45 points)

1.1 Data import (4 points)

1. Solution.

```
hmeq = read_csv("home_loan.csv")
```

```
## Rows: 3460 Columns: 13
## -- Column specification -----
## Delimiter: ","
## chr (3): REASON, JOB, STATUS
## dbl (10): LOAN, MORTDUE, VALUE, YOJ, DEROG, DELINQ, CLAGE, NINQ, CLNO, DEBTINC
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
hmeq = drop_na(hmeq)
print(hmeq)
```

```
## # A tibble: 1,769 x 13
##   LOAN MORTDUE VALUE REASON JOB YOJ DEROG DELINQ CLAGE NINQ CLNO
##   <dbl> <dbl> <dbl> <chr> <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 1700 30548 40320 HomeImp Other 9 0 0 101. 1 8
## 2 1800 28502 43034 HomeImp Other 11 0 0 88.8 0 8
## 3 2400 34863 47471 HomeImp Mgr 12 0 0 70.5 1 21
## 4 2900 7750 67996 HomeImp Other 16 3 0 122. 2 8
## 5 2900 61962 70915 DebtCon Mgr 2 0 0 283. 3 37
## 6 3300 130518 164317 DebtCon Other 9 0 6 192. 0 33
## 7 3700 17857 21144 HomeImp Other 5 0 0 130. 1 9
## 8 3900 29896 45960 HomeImp Other 11 0 0 146. 0 14
## 9 4000 26572 31960 HomeImp Office 11 0 0 118. 0 8
## 10 4400 53091 64074 DebtCon Office 14 1 3 89.9 0 17
## # i 1,759 more rows
## # i 2 more variables: DEBTINC <dbl>, STATUS <chr>
```

2. Solution.

```
hmeq = hmeq |> mutate(STATUS = ifelse(STATUS == "bad", 1, 0))
hmeq$STATUS = factor(hmeq$STATUS, levels = c(0,1))
print(hmeq)
```

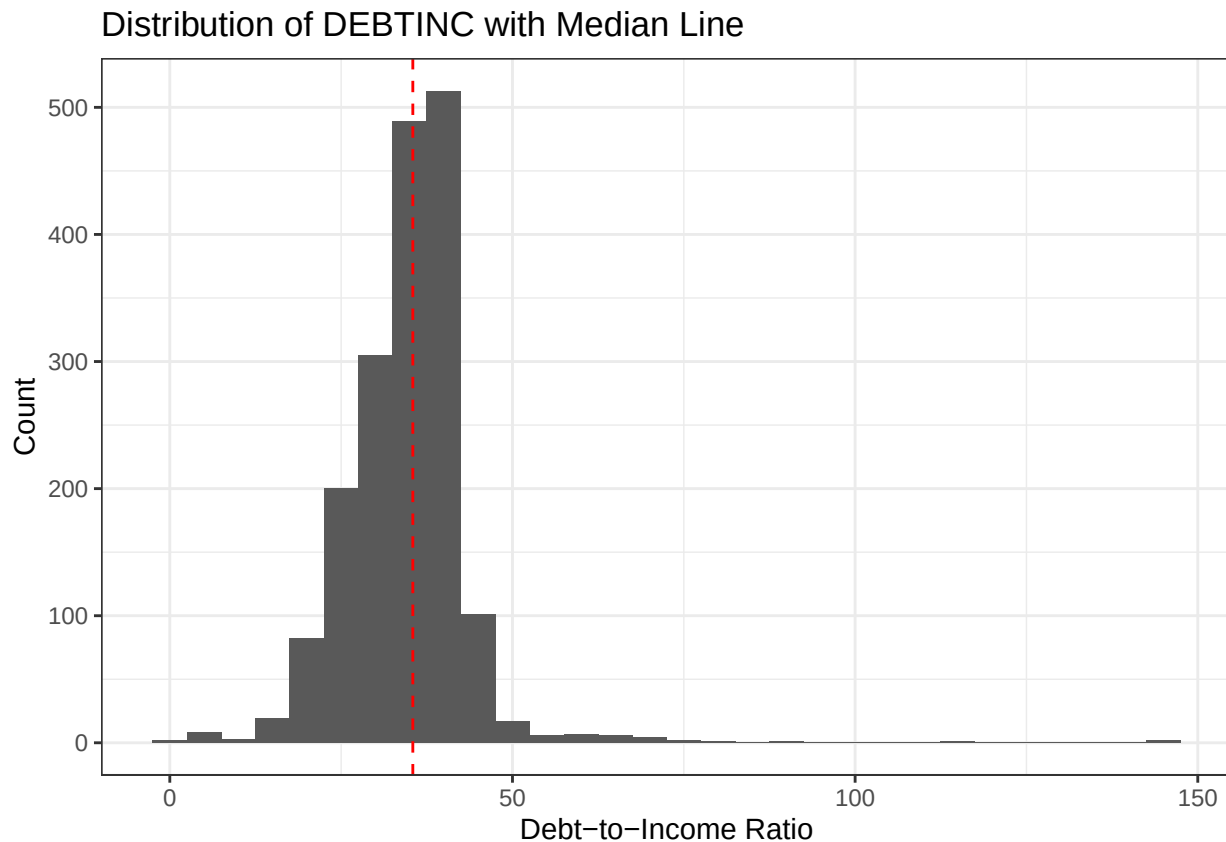
```
## # A tibble: 1,769 x 13
##   LOAN MORTDUE VALUE REASON JOB YOJ DEROG DELINQ CLAGE NINQ CLNO
##   <dbl> <dbl> <dbl> <chr> <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 1700 30548 40320 HomeImp Other 9 0 0 101. 1 8
## 2 1800 28502 43034 HomeImp Other 11 0 0 88.8 0 8
## 3 2400 34863 47471 HomeImp Mgr 12 0 0 70.5 1 21
## 4 2900 7750 67996 HomeImp Other 16 3 0 122. 2 8
## 5 2900 61962 70915 DebtCon Mgr 2 0 0 283. 3 37
## 6 3300 130518 164317 DebtCon Other 9 0 6 192. 0 33
```

```
## 7 3700 17857 21144 HomeImp Other 5 0 0 130. 1 9
## 8 3900 29896 45960 HomeImp Other 11 0 0 146. 0 14
## 9 4000 26572 31960 HomeImp Office 11 0 0 118. 0 8
## 10 4400 53091 64074 DebtCon Office 14 1 3 89.9 0 17
## # i 1,759 more rows
## # i 2 more variables: DEBTINC <dbl>, STATUS <fct>
```

1.2 Data exploration and visualization (4 points)

1. Solution.

```
ggplot(data = hmeq, mapping = aes(x=DEBTINC)) +
  geom_histogram(binwidth = 5) +
  geom_vline(aes(xintercept = median(DEBTINC)), color = "red", linetype = "dashed") +
  labs(title = "Distribution of DEBTINC with Median Line",
       x = "Debt-to-Income Ratio",
       y = "Count")
```

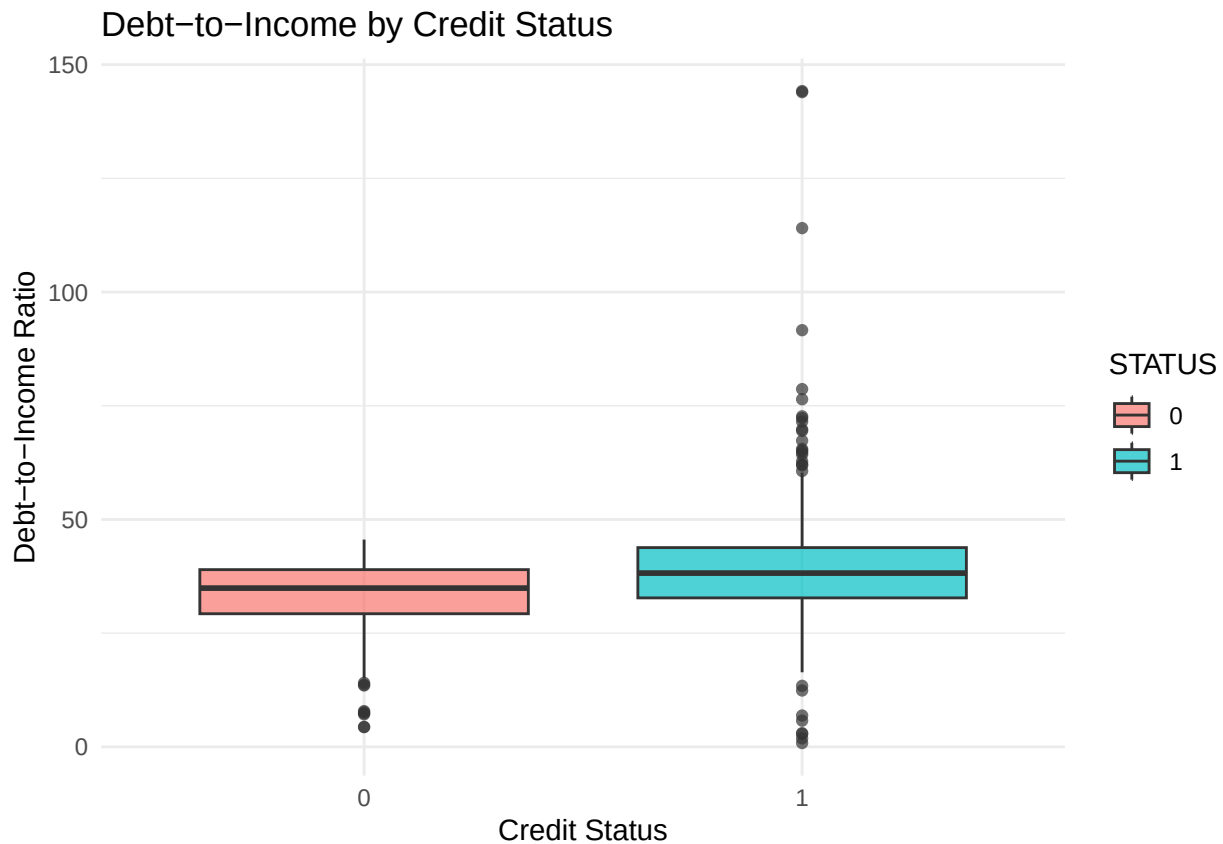


Answer: Based on the histogram, we noticed it's right skewed, where there are some outliers with heavy debt to income ratio.

2. Solution.

```
ggplot(hmeq, aes(x = STATUS, y = DEBTINC, fill = STATUS)) +
  geom_boxplot(alpha = 0.7) +
  labs(title = "Debt-to-Income by Credit Status",
```

```
x = "Credit Status",
y = "Debt-to-Income Ratio") +
theme_minimal()
```



Answer: we notice the paid loans generally have low debt to income ratio and default loans generally have outliers with high debt to income ratio.

1.3 Univariate logistic regression (23 points)

1.3.1 Understanding logistic regression (13 points)

1. Solution.

```
set.seed(4710) # seed set for reproducibility (DO NOT CHANGE)
n <- nrow(hmeq)
train_samples <- sample(1:n, round(0.8 * n))
hmeq_train = hmeq[train_samples,]
hmeq_test = hmeq[-train_samples,]
```

2. Solution.

```
hmeq_train_subset = hmeq_train |>
  filter(VALUE < 50000 & YOJ == 0 & REASON == "DebtCon" & MORTDUE > 23000) |>
  select(STATUS, DEBTINC)
print(hmeq_train_subset)
```

```
## # A tibble: 4 x 2
##   STATUS DEBTINC
##   <fct>   <dbl>
## 1 1      25.3
## 2 0      16.5
## 3 1      29.3
## 4 0      16.7
```

3. **Solution.** In a logistic regression model, the odds of default are given by

$$\text{odds}(\text{default}) = \exp(\beta_0 + \beta_1 \times \text{DEBTINC})$$

Therefore, the probability of default is

$$p(\text{default}) = \frac{\exp(\beta_0 + \beta_1 \times \text{DEBTINC})}{1 + \exp(\beta_0 + \beta_1 \times \text{DEBTINC})}.$$

Given four observations, with two defaulted and two paid loans, the likelihood function is

$$\begin{aligned} L(\beta_0, \beta_1) &= \prod_{i=1}^4 p_i^{y_i} [1 - p_i]^{1-y_i} \\ &= \left[\frac{\exp(\beta_0 + \beta_1 \times 25.3)}{1 + \exp(\beta_0 + \beta_1 \times 25.3)} \right] \times \left[1 - \frac{\exp(\beta_0 + \beta_1 \times 16.5)}{1 + \exp(\beta_0 + \beta_1 \times 16.5)} \right] \times \\ &\quad \left[\frac{\exp(\beta_0 + \beta_1 \times 29.3)}{1 + \exp(\beta_0 + \beta_1 \times 29.3)} \right] \times \left[1 - \frac{\exp(\beta_0 + \beta_1 \times 16.7)}{1 + \exp(\beta_0 + \beta_1 \times 16.7)} \right]. \end{aligned}$$

4. **Solution.**

```
# fit a logistic model based on the subset training data
fit1 = glm(STATUS~DEBTINC, family="binomial", data=hmeq_train_subset)

# identify beta_0 be -115.38, beta_1 be 5.47
summary(fit1)

##
## Call:
## glm(formula = STATUS ~ DEBTINC, family = "binomial", data = hmeq_train_subset)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -115.38   270307.30      0      1
## DEBTINC         5.47   12308.45      0      1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 5.5452e+00  on 3  degrees of freedom
## Residual deviance: 2.6241e-10  on 2  degrees of freedom
## AIC: 4
##
## Number of Fisher Scoring iterations: 23
```

Answer:

The estimated probability of default, given a debt-to-income ratio x , is

$$P(\text{STATUS} = 1 \mid \text{DEBTINC} = x) = \frac{\exp(-115.38 + 5.47x)}{1 + \exp(-115.38 + 5.47x)}.$$

5. **Solution.** Fitted values of β_0 and β_1 are obtained by maximizing the likelihood (and log likelihood) function formulated in part 3.

1.3.2 Univariate logistic regression on the full data (10 points)

1. Solution.

```
# fit a logistic model using the whole training dataset
fit_2 = glm(STATUS~DEBTINC, family = "binomial", data=hmeq_train)
summary(fit_2)

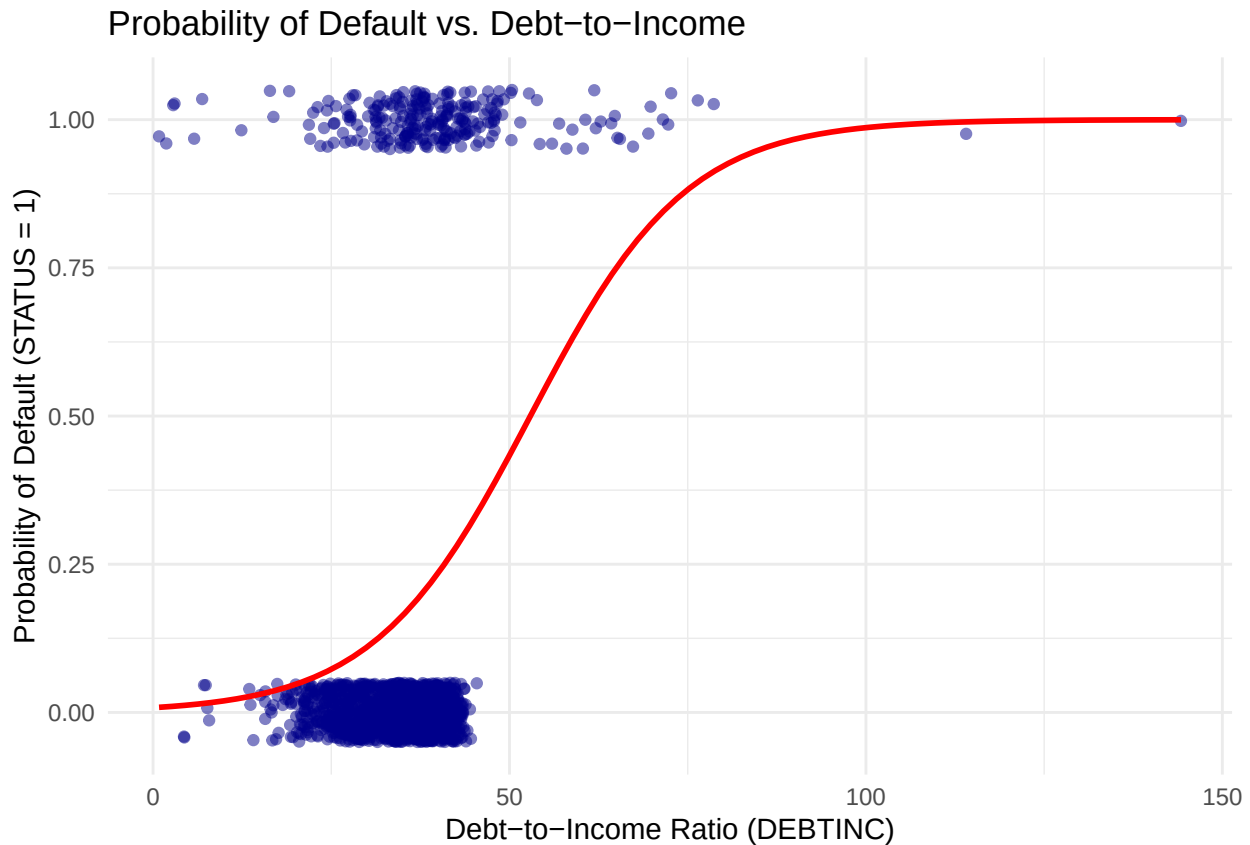
##
## Call:
## glm(formula = STATUS ~ DEBTINC, family = "binomial", data = hmeq_train)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -4.8225     0.3910  -12.33  <2e-16
## DEBTINC       0.0911     0.0103   8.82   <2e-16
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 1328.9  on 1414  degrees of freedom
## Residual deviance: 1224.6  on 1413  degrees of freedom
## AIC: 1229
##
## Number of Fisher Scoring iterations: 5
```

2. **Solution.** Based on the model summary above, we observe for 1 increase in DEBTINC, the odds of default will multiply by $\exp(0.0911)$ (1.095).

3. Solution.

```
ggplot(hmeq_train, aes(x = DEBTINC, y = 1 * (STATUS == 1))) +
  geom_jitter(height = 0.05, width = 0, alpha = 0.5, color = "darkblue") +
  stat_smooth(method = "glm", method.args = list(family = "binomial"), se = FALSE, color = "red", size = 1) +
  labs(
    x = "Debt-to-Income Ratio (DEBTINC)",
    y = "Probability of Default (STATUS = 1)",
    title = "Probability of Default vs. Debt-to-Income"
  ) +
  theme_minimal()

## 'geom_smooth()' using formula = 'y ~ x'
```



Answer: Based on the plot, when DEBTINC is 50, the estimated probability of default is roughly 0.4.

1.4 Multivariate logistic regression (14 points)

1. Solution.

```
fit_3 = glm(STATUS ~ ., family = "binomial", data = hmeq_train)
summary(fit_3)
```

```
##
## Call:
## glm(formula = STATUS ~ ., family = "binomial", data = hmeq_train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -3.95e+00  5.47e-01  -7.22  5.1e-13
## LOAN          -1.87e-05  9.63e-06  -1.94  0.05256
## MORTDUE       -1.50e-06  4.07e-06  -0.37  0.71236
## VALUE         1.48e-06  3.49e-06   0.42  0.67294
## REASONHomeImp -1.15e-02  1.88e-01  -0.06  0.95131
## JOBOffice     -3.50e-01  3.11e-01  -1.13  0.26028
## JOBOther       2.87e-02  2.46e-01   0.12  0.90717
## JOBProfExe     8.47e-02  2.84e-01   0.30  0.76582
## JOBSales       1.17e+00  5.17e-01   2.26  0.02380
## JOBSelf        6.77e-01  4.82e-01   1.40  0.16041
## YOJ           -8.75e-03  1.20e-02  -0.73  0.46740
```



```
## DEROG          7.48e-01  1.28e-01   5.87  4.4e-09
## DELINQ         7.24e-01  8.97e-02   8.07  6.9e-16
## CLAGE         -4.98e-03  1.21e-03  -4.10  4.2e-05
## NINQ          1.54e-01  4.52e-02   3.41  0.00065
## CLNO          -2.17e-02  9.36e-03  -2.32  0.02057
## DEBTINC        9.30e-02  1.15e-02   8.07  7.2e-16
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 1328.9 on 1414 degrees of freedom
## Residual deviance: 1006.3 on 1398 degrees of freedom
## AIC: 1040
##
## Number of Fisher Scoring iterations: 5
```

Answer: Based on the summary, we observe a higher DEBTINC are more likely to default when other things are held equal. And other variables are held equal, for 1 increase in NINQ, it is associated with $\exp(0.154) = 1.17$ times more likely in terms of odds of defaults.

2. Solution.

```
# create a dataframe for the new data
new_data <- data.frame(
  LOAN = 10000,
  MORTDUE = 65000,
  VALUE = 90000,
  REASON = "DebtCon",
  JOB = "Sales",
  YOJ = 5,
  DEROG = 0,
  DELINQ = 0,
  CLAGE = 120,
  NINQ = 1,
  CLNO = 15,
  DEBTINC = 50
)

# predict the new data point
predicted_prob <- predict(fit_3, newdata = new_data, type = "response")

print(predicted_prob)

##      1
## 0.712
```

Answer: There is roughly 71.2% chance of default given above features.

3. Solution.

```
# obtain predicted prob
prob_hat = predict(fit_3, newdata = hmeq_test, type = "response")
```

```

# classify using a 0.5 threshold
pred_class = ifelse(prob_hat >= 0.5, "1", "0")
pred_class = factor(pred_class, levels = c("0","1"))

# calculate four components in confusion matrix
tp = sum(pred_class == "1" & hmeq_test$STATUS == "1")
tn = sum(pred_class == "0" & hmeq_test$STATUS == "0")
fp = sum(pred_class == "1" & hmeq_test$STATUS == "0")
fn = sum(pred_class == "0" & hmeq_test$STATUS == "1")

# calculate metrics
n = nrow(hmeq_test)
misclass_rate = (fp + fn) / n
precision = tp / (tp + fp)
recall = tp / (tp + fn)
F1 = 2 * precision * recall / (precision + recall)

test_errors <- tribble(
  ~Metric, ~Value,
  "Misclassification_rate", misclass_rate,
  "Precision", precision,
  "Recall", recall,
  "F1_score", F1
)
# print the tibble
print(test_errors)

```

```

## # A tibble: 4 x 2
##   Metric                Value
##   <chr>                <dbl>
## 1 Misclassification_rate 0.116
## 2 Precision              0.667
## 3 Recall                0.255
## 4 F1_score              0.369

```

2 Case study: linear regression for housing prices (55 points)

2.1 Exploratory data analysis (10 points)

1. Solution.

```
# read in data
ames_data = read_csv("ames_housing.csv")

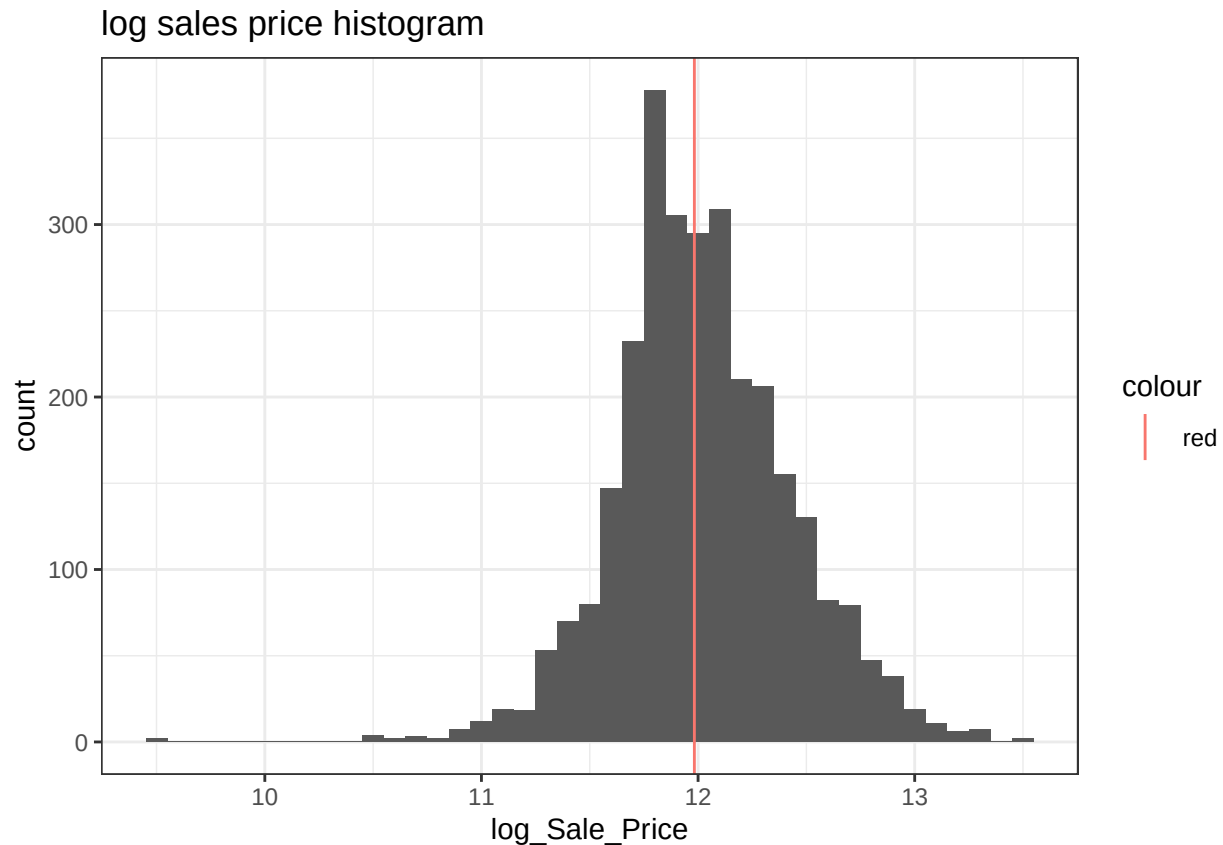
## Rows: 2930 Columns: 173
## -- Column specification -----
## Delimiter: ","
## dbl (173): Lot_Frontage, Lot_Area, Year_Built, Year_Remod_Add, Mas_Vnr_Area,...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.

print(ames_data)

## # A tibble: 2,930 x 173
##   Lot_Frontage Lot_Area Year_Built Year_Remod_Add Mas_Vnr_Area BsmtFin_SF_2
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      141   31770   1960   1960    112      0
## 2       80  11622   1961   1961      0    144
## 3       81  14267   1958   1958    108      0
## 4       93  11160   1968   1968      0      0
## 5       74  13830   1997   1998      0      0
## 6       78   9978   1998   1998     20      0
## 7       41   4920   2001   2001      0      0
## 8       43   5005   1992   1992      0      0
## 9       39   5389   1995   1996      0      0
## 10      60   7500   1999   1999      0      0
## # i 2,920 more rows
## # i 167 more variables: Bsmt_Unf_SF <dbl>, Total_Bsmt_SF <dbl>,
## #   First_Flr_SF <dbl>, Second_Flr_SF <dbl>, Gr_Liv_Area <dbl>,
## #   Bsmt_Full_Bath <dbl>, Bsmt_Half_Bath <dbl>, Full_Bath <dbl>,
## #   Half_Bath <dbl>, Bedroom_AbvGr <dbl>, Kitchen_AbvGr <dbl>,
## #   TotRms_AbvGrd <dbl>, Fireplaces <dbl>, Garage_Cars <dbl>,
## #   Garage_Area <dbl>, Wood_Deck_SF <dbl>, Open_Porch_SF <dbl>, ...
```

2. Solution.

```
# visualize the log price with
ggplot(data = ames_data, aes(x = log_Sale_Price)) +
  geom_histogram(binwidth = 0.1) +
  geom_vline(aes(xintercept = median(log_Sale_Price), color = "red")) +
  labs(title = "log sales price histogram")
```



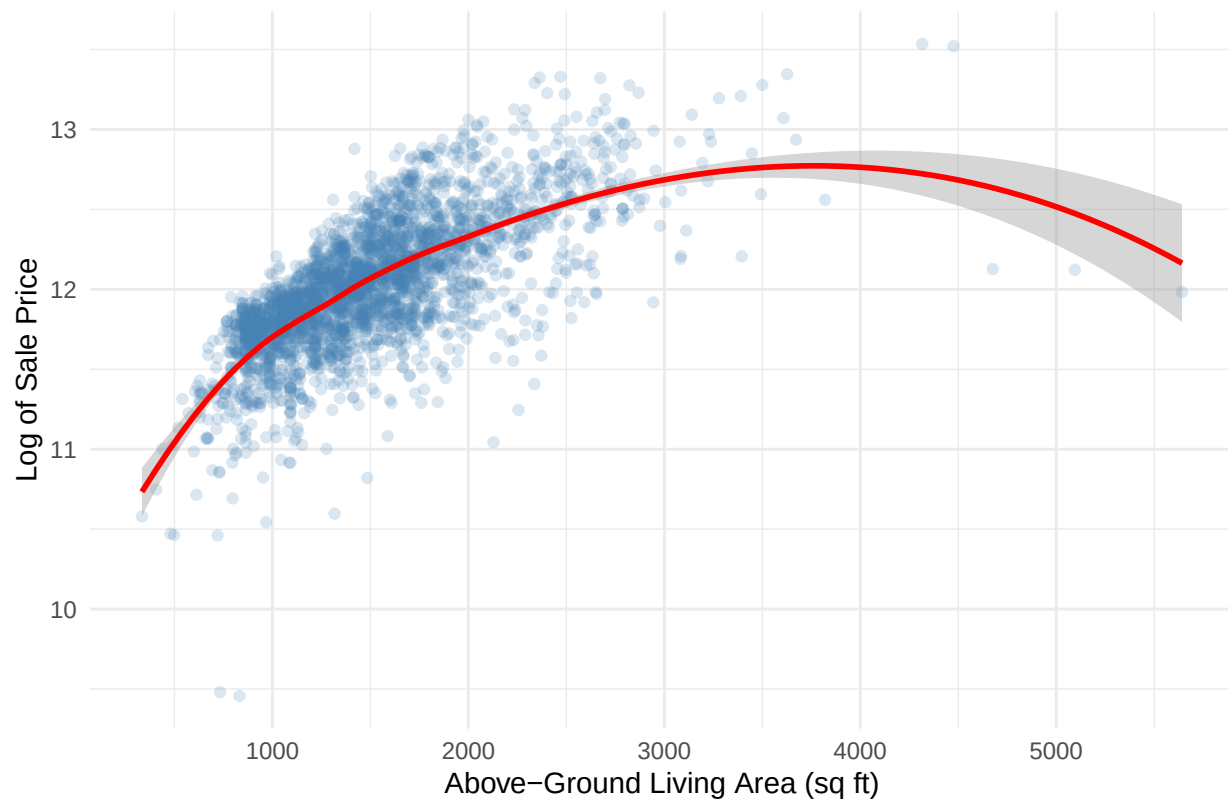
Answer: The distribution of log sales price is roughly symmetric with some outliers at the lower end.

3. Solution.

```
# Scatter plot 1: log_Sale_Price vs. Gr_Liv_Area
ggplot(ames_data, aes(x = Gr_Liv_Area, y = log_Sale_Price)) +
  geom_point(alpha = 0.2, color = "steelblue") +
  geom_smooth(method = "loess", se = TRUE, color = "red") +
  labs(
    title = "Log Sale Price vs. Above-Ground Living Area",
    x = "Above-Ground Living Area (sq ft)",
    y = "Log of Sale Price"
  ) +
  theme_minimal()
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

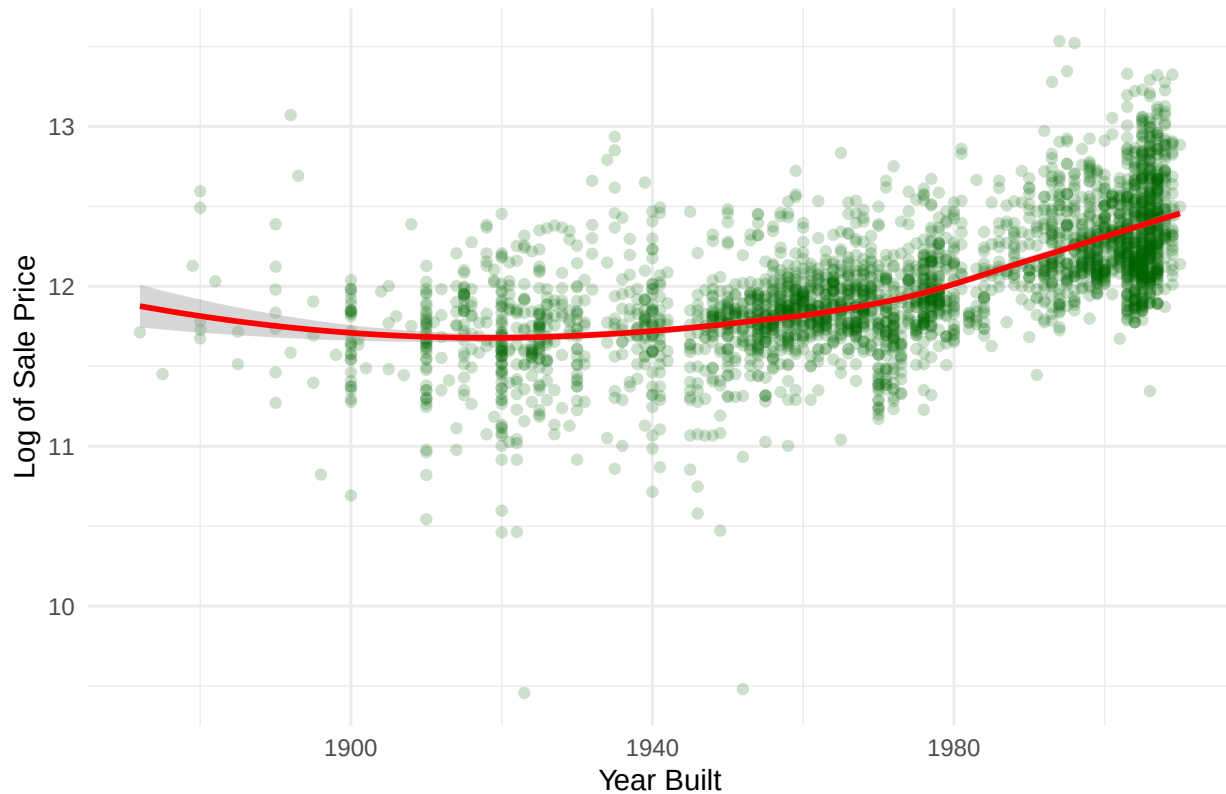
Log Sale Price vs. Above-Ground Living Area



```
# Scatter plot 2: log_Sale_Price vs. Year_Built
ggplot(ames_data, aes(x = Year_Built, y = log_Sale_Price)) +
  geom_point(alpha = 0.2, color = "darkgreen") +
  geom_smooth(method = "loess", se = TRUE, color = "red") +
  labs(
    title = "Log Sale Price vs. Year Built",
    x = "Year Built",
    y = "Log of Sale Price"
  ) +
  theme_minimal()
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

Log Sale Price vs. Year Built



Answer: Based on the two plot above, we observe the log sale price tends to increase with respect to above ground living area before 4000 sq ft then decrease. And the log sales price tends to decrease when year build before 1940s then increase after that.

2.2 Predictive modeling preparation (1 point)

Solution.

```
set.seed(4710) # seed set for reproducibility (DO NOT CHANGE)
n <- nrow(ames_data)
train_samples <- sample(1:n, round(0.8 * n))
ames_train = ames_data[train_samples,]
ames_test = ames_data[-train_samples,]
```

2.3 Ordinary least squares (6 points)

1. **Solution.**

```
ols_fit = lm(log_Sale_Price ~ ., data=ames_train)
summary(ols_fit)

##
## Call:
## lm(formula = log_Sale_Price ~ ., data = ames_train)
##
```

```

## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.1307 -0.0630  0.0002  0.0678  0.6965
##
## Coefficients: (28 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -1.55e+01  1.65e+01  -0.94  0.34661
## Lot_Frontage    1.67e-04  1.01e-04   1.66  0.09620
## Lot_Area        1.41e-06  4.80e-07   2.93  0.00343
## Year_Built      2.66e-03  2.72e-04   9.78 < 2e-16
## Year_Remod_Add  1.06e-03  2.33e-04   4.57  5.1e-06
## Mas_Vnr_Area    3.73e-05  2.57e-05   1.45  0.14779
## BsmtFin_SF_2   -3.11e-05  1.87e-05  -1.66  0.09633
## Bsmt_Unf_SF    -4.37e-05  1.10e-05  -3.97  7.3e-05
## Total_Bsmt_SF   1.31e-04  1.93e-05   6.79  1.5e-11
## First_Flr_SF    1.42e-04  7.24e-05   1.97  0.04928
## Second_Flr_SF   1.54e-04  6.90e-05   2.24  0.02532
## Gr_Liv_Area     1.45e-04  7.14e-05   2.03  0.04230
## Bsmt_Full_Bath  3.54e-02  8.28e-03   4.28  2.0e-05
## Bsmt_Half_Bath -9.72e-03  1.29e-02  -0.75  0.45250
## Full_Bath       3.40e-02  9.10e-03   3.73  0.00020
## Half_Bath       1.19e-02  9.00e-03   1.32  0.18587
## Bedroom_AbvGr  -1.01e-02  5.78e-03  -1.75  0.07953
## Kitchen_AbvGr  -7.21e-02  2.60e-02  -2.77  0.00562
## TotRms_AbvGrd   6.73e-03  4.00e-03   1.68  0.09245
## Fireplaces      4.75e-02  5.72e-03   8.31 < 2e-16
## Garage_Cars     4.91e-02  1.02e-02   4.80  1.7e-06
## Garage_Area     4.19e-05  3.33e-05   1.26  0.20828
## Wood_Deck_SF    1.28e-05  2.60e-05   0.49  0.62135
## Open_Porch_SF   -4.14e-05  4.97e-05  -0.83  0.40516
## Enclosed_Porch  2.18e-04  5.09e-05   4.27  2.0e-05
## Three_season_porch 1.56e-04  1.11e-04   1.40  0.16051
## Screen_Porch    2.66e-04  5.65e-05   4.71  2.6e-06
## Pool_Area       -4.45e-04  3.66e-04  -1.22  0.22411
## Year_Sold       -5.06e-03  2.25e-03  -2.25  0.02428
## Longitude       1.84e-02  1.39e-01   0.13  0.89468
## Latitude        7.40e-01  1.83e-01   4.05  5.4e-05
## Overall_Qual     5.00e-02  3.67e-03  13.61 < 2e-16
## MS_Zoning_A_agr -1.36e+00  2.84e-01  -4.78  1.9e-06
## MS_Zoning_C_all -1.36e-01  3.44e-02  -3.94  8.5e-05
## MS_Zoning_Floating_Village_Residential 1.30e-01  1.82e-02   7.13  1.4e-12
## MS_Zoning_I_all  1.83e-01  1.22e-01   1.50  0.13387
## MS_Zoning_Residential_High_Density 4.92e-02  3.21e-02   1.53  0.12510
## MS_Zoning_Residential_Low_Density 7.32e-02  1.07e-02   6.86  9.0e-12
## MS_Zoning_Residential_Medium_Density NA        NA        NA        NA
## Street_Grvl     -7.48e-02  4.72e-02  -1.59  0.11289
## Street_Pave      NA        NA        NA        NA
## Utilities_AllPub 1.56e-01  1.18e-01   1.32  0.18657
## Utilities_NoSeWa 2.07e-01  1.86e-01   1.11  0.26645
## Utilities_NoSewr NA        NA        NA        NA
## Lot_Config_Corner 6.70e-03  7.91e-03   0.85  0.39732
## Lot_Config_CulDSac 3.87e-02  1.27e-02   3.04  0.00240
## Lot_Config_FR2   -2.95e-02  1.71e-02  -1.73  0.08433
## Lot_Config_FR3   -2.90e-02  4.03e-02  -0.72  0.47179

```

## Lot_Config_Inside	NA	NA	NA	NA
## Land_Slope_Gtl	6.43e-02	4.94e-02	1.30	0.19294
## Land_Slope_Mod	1.00e-01	5.00e-02	2.00	0.04556
## Land_Slope_Sev	NA	NA	NA	NA
## Bldg_Type_Duplex	-5.35e-02	3.05e-02	-1.76	0.07933
## Bldg_Type_OneFam	1.37e-02	2.44e-02	0.56	0.57577
## Bldg_Type_Twnhs	-6.32e-02	3.08e-02	-2.06	0.03998
## Bldg_Type_TwnhsE	5.96e-03	2.79e-02	0.21	0.83084
## Bldg_Type_TwoFmCon	NA	NA	NA	NA
## House_Style_One_and_Half_Fin	4.38e-02	1.36e-02	3.23	0.00124
## House_Style_One_and_Half_Unf	7.31e-02	4.25e-02	1.72	0.08542
## House_Style_One_Story	2.74e-02	1.89e-02	1.45	0.14736
## House_Style_SFoyer	1.12e-02	2.61e-02	0.43	0.66800
## House_Style_SLvl	4.20e-03	2.16e-02	0.19	0.84573
## House_Style_Two_and_Half_Fin	-2.31e-02	6.08e-02	-0.38	0.70445
## House_Style_Two_and_Half_Unf	7.71e-02	3.67e-02	2.10	0.03586
## House_Style_Two_Story	NA	NA	NA	NA
## Roof_Style_Flat	-1.74e-01	1.26e-01	-1.38	0.16702
## Roof_Style_Gable	-6.01e-02	1.06e-01	-0.57	0.56994
## Roof_Style_Gambrel	-4.20e-02	1.11e-01	-0.38	0.70579
## Roof_Style_Hip	-2.91e-02	1.06e-01	-0.27	0.78355
## Roof_Style_Mansard	-1.09e-01	1.23e-01	-0.88	0.37901
## Roof_Style_Shed	NA	NA	NA	NA
## Roof_Matl_ClyTile	-2.96e+00	2.27e-01	-13.03	< 2e-16
## Roof_Matl_CompShg	-4.78e-02	7.02e-02	-0.68	0.49558
## Roof_Matl_Membran	3.03e-01	1.76e-01	1.72	0.08618
## Roof_Matl_Metal	1.91e-01	1.76e-01	1.09	0.27768
## Roof_Matl_Roll	-3.83e-02	1.58e-01	-0.24	0.80837
## Roof_Matl_Tar.Grv	5.05e-02	8.99e-02	0.56	0.57424
## Roof_Matl_WdShake	-6.23e-02	9.19e-02	-0.68	0.49816
## Roof_Matl_WdShngl	NA	NA	NA	NA
## Mas_Vnr_Type_BrkCmn	-9.81e-02	3.52e-02	-2.79	0.00537
## Mas_Vnr_Type_BrkFace	-3.59e-02	1.23e-02	-2.92	0.00353
## Mas_Vnr_Type_CBlock	-3.84e-01	1.78e-01	-2.16	0.03107
## Mas_Vnr_Type_None	-4.18e-02	1.33e-02	-3.15	0.00164
## Mas_Vnr_Type_Stone	NA	NA	NA	NA
## Exter_Cond_Excellent	-4.16e-03	4.96e-02	-0.08	0.93315
## Exter_Cond_Fair	-9.27e-02	2.18e-02	-4.26	2.2e-05
## Exter_Cond_Good	-1.36e-02	1.02e-02	-1.34	0.18192
## Exter_Cond_Poor	-7.58e-02	1.43e-01	-0.53	0.59531
## Exter_Cond_Typical	NA	NA	NA	NA
## Foundation_BrkTil	5.42e-02	8.21e-02	0.66	0.50939
## Foundation_CBlock	1.64e-02	8.12e-02	0.20	0.83995
## Foundation_PConc	7.18e-02	8.09e-02	0.89	0.37447
## Foundation_Slab	-2.32e-02	8.93e-02	-0.26	0.79483
## Foundation_Stone	1.77e-01	9.78e-02	1.81	0.07107
## Foundation_Wood	NA	NA	NA	NA
## Bsmt_Exposure_Av	3.94e-02	3.57e-02	1.10	0.27030
## Bsmt_Exposure_Gd	8.67e-02	3.68e-02	2.36	0.01850
## Bsmt_Exposure_Mn	1.11e-02	3.56e-02	0.31	0.75480
## Bsmt_Exposure_No	-4.93e-03	3.41e-02	-0.14	0.88523
## Bsmt_Exposure_No_Basement	NA	NA	NA	NA
## Heating_Floor	NA	NA	NA	NA
## Heating_GasA	-6.13e-02	7.02e-02	-0.87	0.38299

## Heating_GasW	2.39e-03	7.67e-02	0.03	0.97512
## Heating_Grav	-2.28e-01	8.80e-02	-2.59	0.00962
## Heating_OthW	-4.38e-02	1.23e-01	-0.36	0.72141
## Heating_Wall	NA	NA	NA	NA
## Heating_QC_Excellent	3.48e-02	8.69e-03	4.01	6.3e-05
## Heating_QC_Fair	1.21e-02	1.89e-02	0.64	0.52362
## Heating_QC_Good	1.01e-02	9.51e-03	1.06	0.28990
## Heating_QC_Poor	-9.99e-02	1.03e-01	-0.97	0.33175
## Heating_QC_Typical	NA	NA	NA	NA
## Central_Air_N	-5.27e-02	1.63e-02	-3.24	0.00123
## Central_Air_Y	NA	NA	NA	NA
## Electrical_FuseA	-6.03e-02	1.39e-01	-0.43	0.66521
## Electrical_FuseF	-6.73e-02	1.41e-01	-0.48	0.63278
## Electrical_FuseP	-1.38e-01	1.51e-01	-0.91	0.36077
## Electrical_Mix	2.16e-01	2.11e-01	1.02	0.30686
## Electrical_SBrkr	-7.63e-02	1.39e-01	-0.55	0.58234
## Electrical_Unknown	NA	NA	NA	NA
## Functional_Maj1	-1.28e-01	3.63e-02	-3.53	0.00042
## Functional_Maj2	-2.62e-01	5.84e-02	-4.48	7.9e-06
## Functional_Min1	-5.26e-02	2.01e-02	-2.61	0.00902
## Functional_Min2	-5.09e-02	1.98e-02	-2.57	0.01027
## Functional_Mod	-7.65e-02	2.88e-02	-2.66	0.00795
## Functional_Sal	-2.51e-01	1.44e-01	-1.75	0.08040
## Functional_Sev	-2.07e-01	1.04e-01	-1.99	0.04679
## Functional_Typ	NA	NA	NA	NA
## Garage_Type_Attchd	3.53e-02	1.04e-01	0.34	0.73368
## Garage_Type_Basment	-2.66e-02	1.08e-01	-0.25	0.80443
## Garage_Type_BuiltIn	5.41e-03	1.05e-01	0.05	0.95876
## Garage_Type_CarPort	-5.23e-02	1.11e-01	-0.47	0.63628
## Garage_Type_Detchd	1.92e-02	1.03e-01	0.19	0.85296
## Garage_Type_More_Than_Two_Types	1.42e-03	1.09e-01	0.01	0.98958
## Garage_Type_No_Garage	NA	NA	NA	NA
## Garage_Finish_Fin	1.00e-02	1.04e-02	0.96	0.33626
## Garage_Finish_No_Garage	1.44e-02	1.02e-01	0.14	0.88838
## Garage_Finish_RFn	-1.66e-03	9.43e-03	-0.18	0.86020
## Garage_Finish_Unf	NA	NA	NA	NA
## Garage_Cond_Excellent	7.25e-02	8.03e-02	0.90	0.36636
## Garage_Cond_Fair	-6.11e-02	1.99e-02	-3.06	0.00223
## Garage_Cond_Good	-2.90e-03	3.78e-02	-0.08	0.93886
## Garage_Cond_No_Garage	NA	NA	NA	NA
## Garage_Cond_Poor	-2.16e-02	5.14e-02	-0.42	0.67457
## Garage_Cond_Typical	NA	NA	NA	NA
## Paved_Drive_Dirt_Gravel	-4.49e-02	1.44e-02	-3.12	0.00186
## Paved_Drive_Partial_Pavement	-1.25e-02	2.05e-02	-0.61	0.54334
## Paved_Drive_Paved	NA	NA	NA	NA
## Pool_QC_Excellent	1.26e-01	1.16e-01	1.08	0.28031
## Pool_QC_Fair	-2.03e-02	1.39e-01	-0.15	0.88367
## Pool_QC_Good	3.24e-01	1.78e-01	1.82	0.06930
## Pool_QC_No_Pool	-1.54e-01	1.86e-01	-0.83	0.40642
## Pool_QC_Typical	NA	NA	NA	NA
## Fence_Good_Privacy	2.68e-03	1.49e-02	0.18	0.85728
## Fence_Good_Wood	-2.14e-02	1.55e-02	-1.38	0.16812
## Fence_Minimum_Privacy	-1.11e-02	9.74e-03	-1.14	0.25630
## Fence_Minimum_Wood_Wire	-1.26e-04	4.99e-02	0.00	0.99798

```
## Fence_No_Fence          NA          NA          NA          NA
## Sale_Type_COD          -3.44e-03    2.03e-02    -0.17    0.86543
## Sale_Type_Con          9.27e-02    6.33e-02     1.46    0.14354
## Sale_Type_ConLD        1.83e-02    3.23e-02     0.57    0.56991
## Sale_Type_ConLI       -1.19e-02    5.01e-02    -0.24    0.81187
## Sale_Type_ConLw       -2.30e-03    5.36e-02    -0.04    0.96581
## Sale_Type_CWD          3.74e-02    4.21e-02     0.89    0.37509
## Sale_Type_New          6.24e-02    6.59e-02     0.95    0.34371
## Sale_Type_Oth         -3.05e-02    5.72e-02    -0.53    0.59393
## Sale_Type_VWD         -1.39e-01    1.39e-01    -1.00    0.31861
## Sale_Type_WD           NA          NA          NA          NA
## Sale_Condition_Abnorml  -6.97e-02    6.60e-02    -1.06    0.29129
## Sale_Condition_AdjLand  1.17e-01    8.05e-02     1.46    0.14535
## Sale_Condition_Alloca   2.32e-02    7.43e-02     0.31    0.75531
## Sale_Condition_Family   1.57e-02    6.88e-02     0.23    0.82007
## Sale_Condition_Normal   2.37e-02    6.50e-02     0.36    0.71518
## Sale_Condition_Partial  NA          NA          NA          NA
##
## Residual standard error: 0.136 on 2199 degrees of freedom
## Multiple R-squared:  0.895, Adjusted R-squared:  0.888
## F-statistic: 130 on 144 and 2199 DF, p-value: <2e-16
```

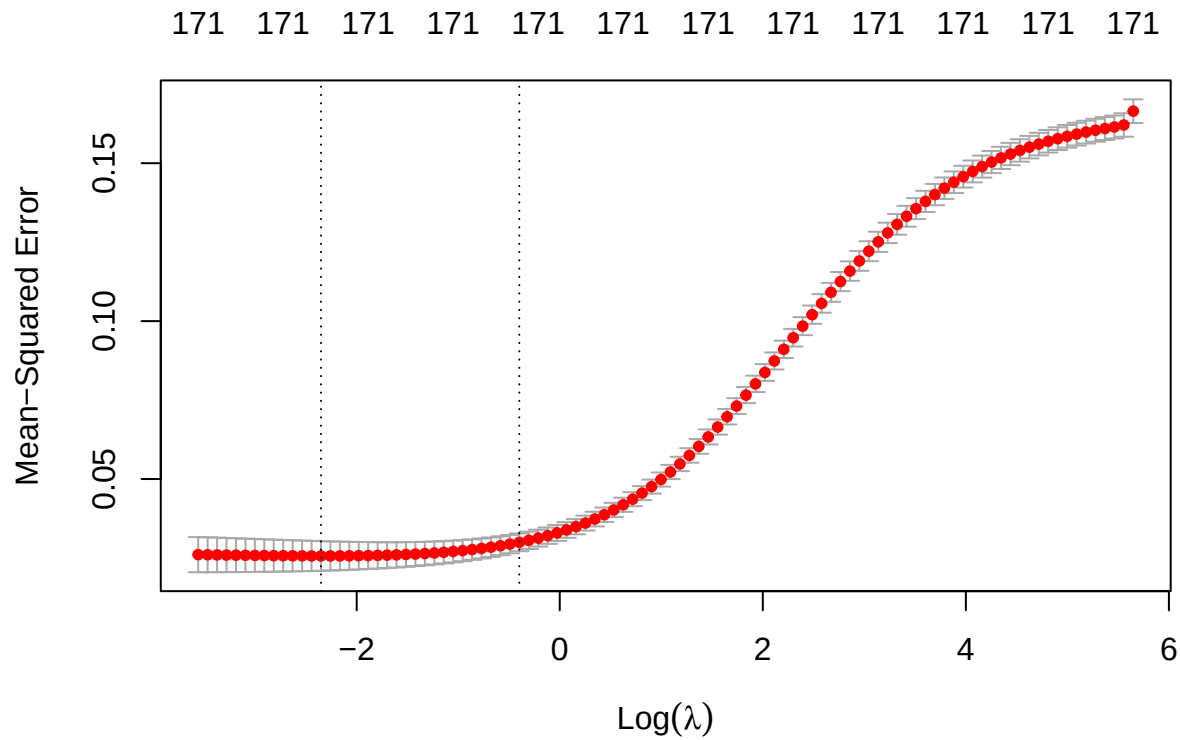
2. **Solution.** Some NA values are in the fitted coefficients because some features are linearly dependent with others. For instance, some categorical variables, like sale condition, are not entered as factors (of which R will automatically drop one category), hence resulting the last category linearly dependent on other categories in the same variable.
3. **Solution.** Despite both signs of Gr_Liv_Area and Year_Built with respect to log Sale Price are positive, those two values are close to zero (2.66e-3 for Year Build and 1.45e-4 for Gr_live_area), which does not match the patterns we see in previous two plots.

2.4 Ridge regression (13 points)

1. Solution.

```
set.seed(471)
ridge_fit <- cv.glmnet(
  log_Sale_Price ~ .,
  alpha = 0,           # alpha = 0 for ridge (stay tuned for more)
  nfolds = 10,         # number of folds
  data = ames_train    # data to run ridge on
)

# plot the cv mean error as a function of lambda
plot(ridge_fit)
```



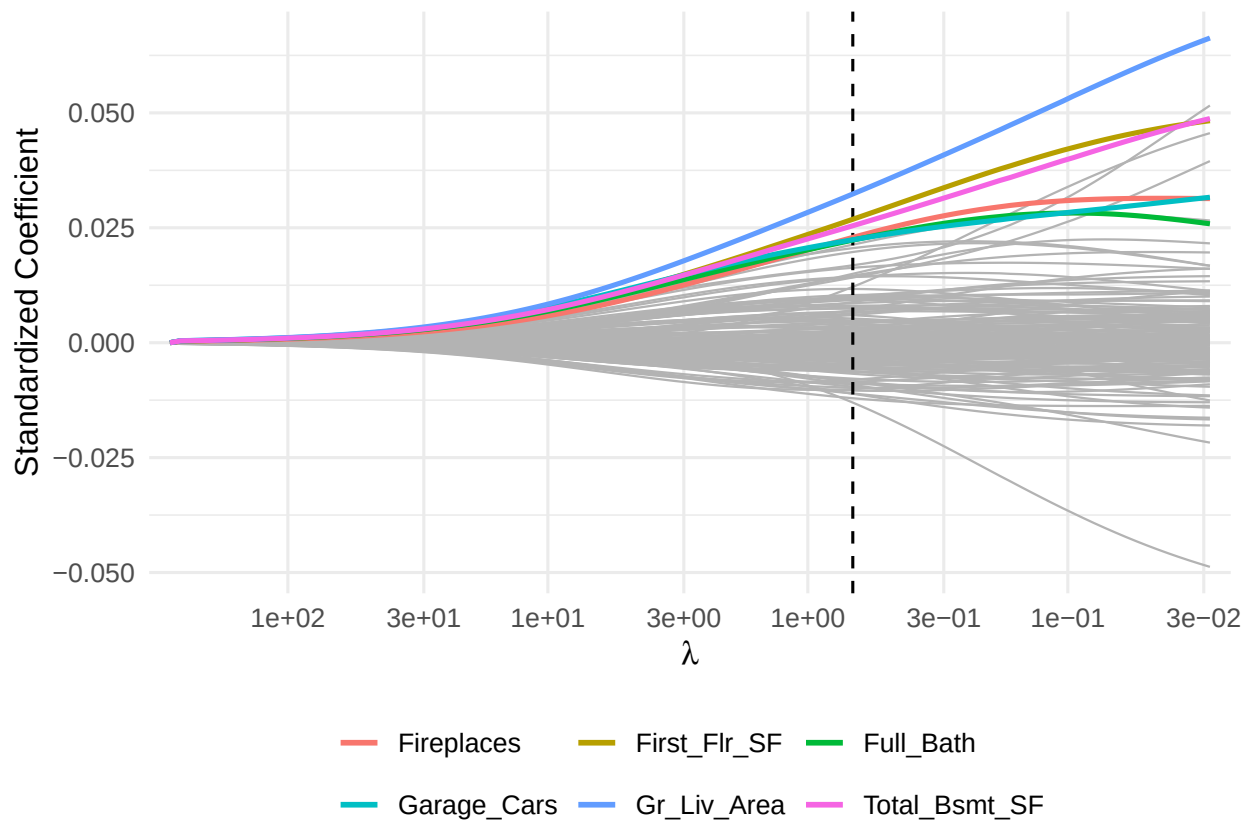
```
# identify 1se for lambda
ridge_fit$lambda.1se
```

```
## [1] 0.672
```

Answer: The lambda for 1 SE rule is 0.672.

2. **Solution. Answer:** Since each fold we have 100 fit on for each value of lambda, in total we have $10 * 100 = 1000$ model fit excluding the final fit on the whole data.
3. **Solution. Answer:** Since each data is used in 9 training fold and 1 validation fold, each data is used $9 * 100 = 900$ times using it.
4. **Solution.**

```
# visualize ridge coefs
plot_glmnet(ridge_fit, ames_train, features_to_plot = 6)
```



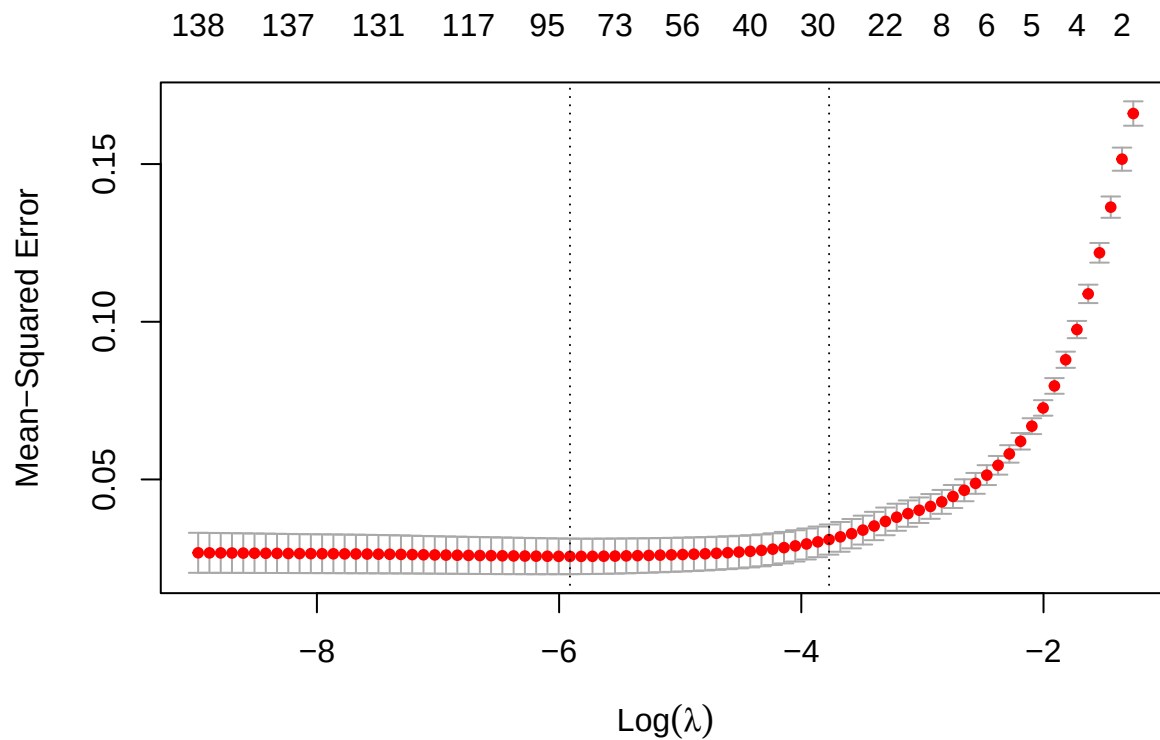
Answer: None of the 6 features change sign as lambda decrease and only some of the 6 features change monotonically as lambda decreases.

2.5 Lasso regression (10 points)

1. Solution.

```
set.seed(471)
lasso_fit <- cv.glmnet(
  log_Sale_Price ~ .,
  alpha = 1,           # alpha = 1 for lasso
  nfolds = 10,         # num of folds
  data = ames_train    # data to run on
)

# plot the cv mean error as a function of lambda
plot(lasso_fit)
```



lution.

2. So-

```
# check 1 se lambda
print(lasso_fit$lambda.1se)
```

```
## [1] 0.023
```

```
# extract 1se lambda coefs
beta_1se = as.matrix(coef(lasso_fit, s = lasso_fit$lambda.1se))

# remove the intercept
beta_no_intercept = beta_1se[-1, , drop = FALSE]

# identify nonzero and zero coefficients
nz_idx = which(beta_no_intercept[, 1] != 0)
z_idx = which(beta_no_intercept[, 1] == 0)

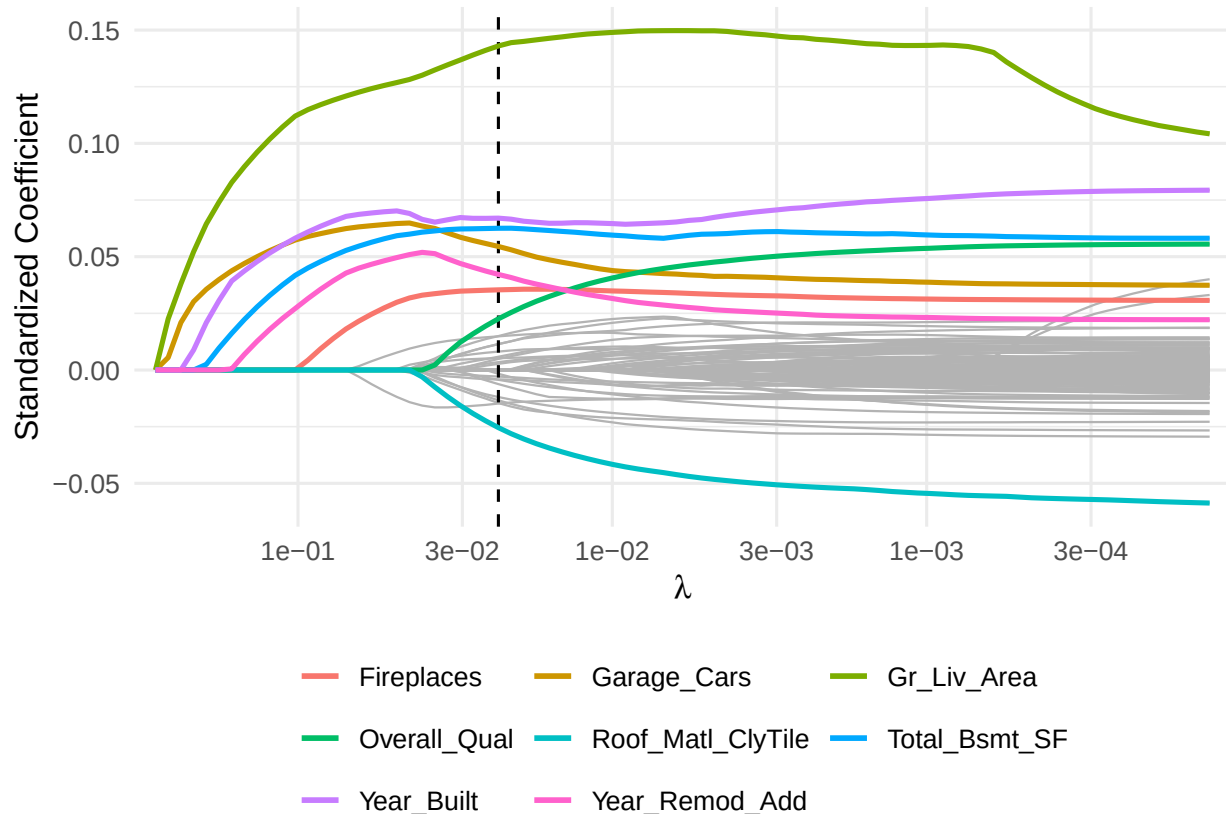
# Compute counts
num_nonzero = length(nz_idx)
print(num_nonzero)
```

```
## [1] 27
```

Answer: There are 27 features have nonzero coefficients in the Lasso fit with 1 se lambda, which is 0.023.

3. Solution.

```
# visualize lasso coeffs
plot_glmnet(lasso_fit, ames_train, features_to_plot = 8)
```



Answer: Based on the plot, Gr_Liv_Area is the first feature to enter the model as lambda decreases and the same feature (Gr_Liv_Area) has the largest absolute coefficient for the most flexibly fitted Lasso.

2.6 Evaluation and comparison (15 points)

1. Solution.

```
# get three models predictions
ols_predictions = predict(ols_fit, newdata = ames_test) |> as.numeric()

ridge_predictions = predict(ridge_fit,
  newdata = ames_test,
  s = "lambda.1se"
) |> as.numeric()

lasso_predictions = predict(lasso_fit,
  newdata = ames_test,
  s = "lambda.1se"
) |> as.numeric()

# extract y label
y_test = ames_test$log_Sale_Price

# compute mse
```

```

mse_ols = mean((y_test - ols_predictions)^2)
mse_ridge = mean((y_test - ridge_predictions)^2)
mse_lasso = mean((y_test - lasso_predictions)^2)

error_for_models <- tribble(
  ~Model, ~RMSE,
  "Linear", mse_ols,
  "Ridge", mse_ridge,
  "Lasso", mse_lasso
)
# print the tibble
print(error_for_models)

```

```

## # A tibble: 3 x 2
##   Model    RMSE
##   <chr>   <dbl>
## 1 Linear 0.0202
## 2 Ridge 0.0274
## 3 Lasso 0.0244

```

2. **Solution. Answer** Based on the above result, we observe the OLS has the lowest RMSE compared with Ridge and Lasso regression. Since regularization models like Ridge and Lasso are used to decrease variance and increase bias, this indicates bias is the dominant driving force for test error. This makes sense, since our whole dataset has 2930 data points, which is fairly large dataset for 172 features.

3. Solution.

```

# extracting coeffs
ls_coef = coef(ols_fit) |> as.numeric()
ridge_coef = coef(ridge_fit, s = "lambda.1se") |> as.matrix()
lasso_coef = coef(lasso_fit, s = "lambda.1se") |> as.matrix()

# get feature names
feature_names = rownames(ridge_coef)[-1]
ls_coef_values = ls_coef[-1]
ridge_coef_values = ridge_coef[-1, 1]
lasso_coef_values = lasso_coef[-1, 1]

# create a coef tibble
coeffs = tibble(
  feature = feature_names,
  ls_coef = ls_coef_values,
  ridge_coef = ridge_coef_values,
  lasso_coef = lasso_coef_values
) |>
  arrange(desc(abs(ls_coef)))

# print tibble
print(coeffs)

```

```

## # A tibble: 172 x 4
##   feature          ls_coef ridge_coef lasso_coef

```

```
##      <chr>                <dbl>      <dbl>      <dbl>
## 1 Roof_Matl_ClyTile      -2.96      -0.631     -1.23
## 2 MS_Zoning_A_agr        -1.36      -0.410     -0.648
## 3 Latitude                0.740       0.534       0
## 4 Mas_Vnr_Type_CBlock    -0.384     -0.144       0
## 5 Pool_QC_Good           0.324     -0.107       0
## 6 Roof_Matl_Membran      0.303       0.0783      0
## 7 Functional_Maj2        -0.262     -0.0988      0
## 8 Functional_Sal         -0.251     -0.289     -0.0992
## 9 Heating_Grav          -0.228     -0.0654      0
## 10 Electrical_Mix        0.216     -0.0332      0
## # i 162 more rows
```

Answer: we notice the coefficients magnitude from OLS is always larger than Ridge model, since OLS is the case for lambda be zero for Ridge model.

4. Solution.

```
# get rows for ls coef be na
na_rows = coeffs |> filter(is.na(ls_coef))
print(na_rows)
```

```
## # A tibble: 28 x 4
##   feature                                ls_coef ridge_coef lasso_coef
##   <chr>                                <dbl>      <dbl>      <dbl>
## 1 MS_Zoning_Residential_Medium_Density NA    -0.0300    -0.0397
## 2 Street_Pave                        NA     0.0449      0
## 3 Utilities_NoSewr                   NA    -0.0551      0
## 4 Lot_Config_Inside                   NA    -0.00678     0
## 5 Land_Slope_Sev                      NA    -0.00531     0
## 6 Bldg_Type_TwoFmCon                   NA    -0.0126      0
## 7 House_Style_Two_Story                 NA     0.0115      0
## 8 Roof_Style_Shed                      NA   -0.000895     0
## 9 Roof_Matl_WdShngl                    NA     0.0792      0
## 10 Mas_Vnr_Type_Stone                   NA     0.0315      0
## # i 18 more rows
```

Answer: For the OLS NA fitted coefficients, the ridge and lasso models do not have them be NA. Ridge addresses the issue by assigning dependent features similar coefficients values and Lasso addresses the issue by randomly assigning one of the dependent features 0 coefficients.